Statistics Maintenance Framework

After repeated TempDB disk latency alerts during scheduled statistics maintenance, I reviewed the maintenance design and found that blanket statistics updates were creating unnecessary load across tables that did not need the same level of refresh. Blanket statistics updates can introduce unnecessary I/O, CPU usage, and TempDB pressure, particularly in environments with large or infrequently modified tables. This framework outlines a phased approach for identifying high-impact tables and shifting to a more targeted statistics maintenance model to reduce TempDB pressure, shorten maintenance windows, and retain optimizer benefits where they matter most.

Part 1 - Investigation

First we need to identify what tables we want to target first and then prioritize those for our maintenance tasks.

This query helps identify the most resource-intensive tables within a database by table size, row count, number of stats, number of indexes and provides a weight to determine candidates for targeted statistics update maintenance jobs. This query needs to be run on each user database you want to target tables in, so you'll need to run this multiple times to gather your lists.

You're looking for tables classified as "**VERY HEAVY**" and "**HEAVY**" which are definite candidates for ongoing targeted statistics Maintenance but for ones classified as "**MODERATE**" evaluate tables with high "**total_modification_counter**" and "**stats_count**" as targets and select top 10 or so of each to mark for an additional pass.

You'll create 2 separate lists, List 1 - will be for VERY HEAVY and HEAVY tables, List 2 - will be for targeted MODERATE tables.

An additional note, you'll want to re-run these evaluations on occasion as depending on your environment, table usage can change at various times, or tables once LIGHT or MODERATE may become more HEAVY as time goes on.

```
USE YourDatabaseName;
GO

SET NOCOUNT ON;

IF OBJECT_ID('tempdb..#HeavyTableAssessment') IS NOT NULL
   DROP TABLE #HeavyTableAssessment;

WITH table_size AS
(
   SELECT
      t.object_id,
      s.name AS schema_name,
      t.name AS table_name,
      SUM(CASE WHEN p.index_id IN (0,1) THEN p.rows ELSE 0 END) AS row_count,
      CAST(SUM(a.total_pages) * 8.0 / 1024.0 AS DECIMAL(18,2)) AS total_mb,
      CAST(SUM(a.used_pages) * 8.0 / 1024.0 AS DECIMAL(18,2)) AS used_mb
   FROM sys.tables t
   JOIN sys.schemas s
      ON t.schema_id = s.schema_id
   JOIN sys.indexes i
      ON t.object_id = i.object_id
   JOIN sys.partitions p
      ON i.object_id = p.object_id
     AND i.index_id = p.index_id
   JOIN sys.allocation_units a
      ON p.partition_id = a.container_id
   WHERE t.is_ms_shipped = 0
   GROUP BY
      t.object_id,
      s.name,
      t.name
),
stats_info AS
(
   SELECT
      st.object_id,
```

```
    COUNT(*) AS stats_count,
    SUM(CASE WHEN st.auto_created = 1 THEN 1 ELSE 0 END) AS auto_stats_count,
    MAX(ISNULL(sp.modification_counter,0)) AS max_modification_counter,
    SUM(ISNULL(sp.modification_counter,0)) AS total_modification_counter,
    MAX(sp.last_updated) AS last_stats_update
  FROM sys.stats st
  OUTER APPLY sys.dm_db_stats_properties(st.object_id, st.stats_id) sp
  WHERE OBJECTPROPERTY(st.object_id, 'IsUserTable') = 1
  GROUP BY st.object_id
),
index_info AS
(
  SELECT
    object_id,
    COUNT(*) AS index_count
  FROM sys.indexes
  WHERE index_id > 0
  GROUP BY object_id
),
base AS
(
  SELECT
    DB_NAME() AS database_name,
    ts.object_id,
    ts.schema_name,
    ts.table_name,
    ISNULL(ts.row_count,0) AS row_count,
    ISNULL(ts.total_mb,0) AS total_mb,
    ISNULL(ts.used_mb,0) AS used_mb,
    ISNULL(ii.index_count,0) AS index_count,
    ISNULL(si.stats_count,0) AS stats_count,
    ISNULL(si.auto_stats_count,0) AS auto_stats_count,
    ISNULL(si.max_modification_counter,0) AS max_modification_counter,
    ISNULL(si.total_modification_counter,0) AS total_modification_counter,
    si.last_stats_update
  FROM table_size ts
  LEFT JOIN stats_info si
    ON ts.object_id = si.object_id
  LEFT JOIN index_info ii
    ON ts.object_id = ii.object_id
),
scored AS
(
  SELECT
    *,
    CAST(
        (CASE
            WHEN total_mb >= 102400 THEN 40   -- 100 GB+
            WHEN total_mb >= 51200  THEN 35   -- 50 GB+
            WHEN total_mb >= 10240  THEN 28   -- 10 GB+
            WHEN total_mb >= 5120   THEN 22   -- 5 GB+
            WHEN total_mb >= 1024   THEN 15   -- 1 GB+
            WHEN total_mb >= 256    THEN 8
            ELSE 2
          END)
        + (CASE
            WHEN row_count >= 100000000 THEN 25
            WHEN row_count >= 50000000  THEN 20
            WHEN row_count >= 10000000  THEN 15
            WHEN row_count >= 1000000   THEN 10
            WHEN row_count >= 100000    THEN 5
            ELSE 1
          END)
        + (CASE
            WHEN stats_count >= 100 THEN 15
```

```
            WHEN stats_count >= 50  THEN 10
            WHEN stats_count >= 20  THEN 6
            WHEN stats_count >= 10  THEN 3
            ELSE 1
          END)
        + (CASE
            WHEN index_count >= 20 THEN 10
            WHEN index_count >= 10 THEN 7
            WHEN index_count >= 5  THEN 4
            ELSE 1
          END)
        + (CASE
            WHEN total_modification_counter >= 10000000 THEN 20
            WHEN total_modification_counter >= 1000000  THEN 15
            WHEN total_modification_counter >= 100000   THEN 10
            WHEN total_modification_counter >= 10000    THEN 5
            ELSE 1
          END)
      AS INT) AS heavy_score
  FROM base
)
SELECT
  database_name,
  schema_name,
  table_name,
  row_count,
  total_mb,
  used_mb,
  index_count,
  stats_count,
  auto_stats_count,
  max_modification_counter,
  total_modification_counter,
  last_stats_update,
  heavy_score,
  CASE
    WHEN heavy_score >= 90 THEN 'VERY HEAVY'
    WHEN heavy_score >= 65 THEN 'HEAVY'
    WHEN heavy_score >= 40 THEN 'MODERATE'
    ELSE 'LIGHT'
  END AS heaviness_band
INTO #HeavyTableAssessment
FROM scored;

SELECT TOP (50)
  database_name,
  schema_name,
  table_name,
  row_count,
  total_mb,
  used_mb,
  index_count,
  stats_count,
  auto_stats_count,
  max_modification_counter,
  total_modification_counter,
  last_stats_update,
  heavy_score,
  heaviness_band
FROM #HeavyTableAssessment
ORDER BY heavy_score DESC, total_mb DESC, row_count DESC;
```

Part 2 - Statistics Maintenance Job Setup

Now that we have our lists created and filtered, set up the SQL Agent Maintenance jobs. We will be using 2 separate jobs. This will help lessen the load on the server, and help this overall task run more efficiently and helps with management and troubleshooting (if needed). First Pass runs MODERATE or LIGHT table updates as those will update faster, and afterwards the Second Pass does VERY HEAVY or HEAVY tables which can take a little longer.

There will be 2 job sets (a set is both First Pass and Second Pass jobs) for Daily statistics Updates, which updates statistics using a 10% sample, and the second set will be for Monthly statistics Updates which will do a FULLSCAN on all the defined target tables.

Below is an example of the First Pass SQL Job setup scripts, which can be run to create the First Pass jobs for each. You'll want to enter in your targeted databases in the "ADD YOUR TARGET TABLES HERE" section of the script, and you'll want to modify the schedule for your environment's maintenance window.

You will need to then take these scripts, and modify them to setup your Second Pass SQL Job, same changes as the First Pass job, but additionally you'll want to of course change the Job/Schedule/Step names from first pass to second pass and you'll want to set the schedule to say 15-30 minutes or any time you desire after the First Pass has completed.

a.) Daily Statistics Update Job Script

```
USE msdb;
GO

SET NOCOUNT ON;
GO

------------------------------------------------------------------------
-- Drop existing job if it already exists
------------------------------------------------------------------------
IF EXISTS
(
   SELECT 1
   FROM msdb.dbo.sysjobs
   WHERE name = N'Daily Update Statistics - First Pass'
)
BEGIN
   EXEC msdb.dbo.sp_delete_job
      @job_name = N'Daily Update Statistics - First Pass',
      @delete_unused_schedule = 1;
END
GO

BEGIN TRANSACTION;
GO

DECLARE @ReturnCode INT = 0;
DECLARE @JobId BINARY(16);
GO

------------------------------------------------------------------------
-- Create Job
------------------------------------------------------------------------
EXEC msdb.dbo.sp_add_job
   @job_name = N'Daily Update Statistics - First Pass',
   @enabled = 1,
   @notify_level_eventlog = 0,
   @notify_level_email = 0,
   @notify_level_netsend = 0,
   @notify_level_page = 0,
   @delete_level = 0,
   @description = N'Performs first-pass targeted daily statistics maintenance using table-level UPDATE STATISTICS WITH SAMPLE 10
PERCENT against a curated list of tables. Intended to reduce broad maintenance impact and support controlled stats refresh on
prioritized objects.',
   @category_name = N'[Uncategorized (Local)]',
   @owner_login_name = SUSER_SNAME(),
   @job_id = @JobId OUTPUT;
```

```
GO

-------------------------------------------------------------------------------
-- Add Job Step
-------------------------------------------------------------------------------
EXEC msdb.dbo.sp_add_jobstep
   @job_name = N'Daily Update Statistics - First Pass',
   @step_name = N'Run targeted SAMPLE 10 PERCENT stats updates',
   @subsystem = N'TSQL',
   @database_name = N'master',
   @on_success_action = 1,
   @on_fail_action = 2,
   @retry_attempts = 0,
   @retry_interval = 0,
   @command = N'
SET NOCOUNT ON;

DECLARE @StatsMode VARCHAR(10) = ''DAILY'';
DECLARE @UpdateClause NVARCHAR(100);

SET @UpdateClause =
   CASE
      WHEN @StatsMode = ''DAILY'' THEN N''WITH SAMPLE 10 PERCENT''
      WHEN @StatsMode = ''MONTHLY'' THEN N''WITH FULLSCAN''
      ELSE NULL
   END;

IF @UpdateClause IS NULL
BEGIN
   RAISERROR(''Invalid @StatsMode. Use DAILY or MONTHLY.'', 16, 1);
   RETURN;
END;

IF OBJECT_ID(''tempdb..#TargetStatsTables'') IS NOT NULL
   DROP TABLE #TargetStatsTables;

CREATE TABLE #TargetStatsTables
(
   database_name SYSNAME NOT NULL,
   schema_name   SYSNAME NOT NULL,
   table_name    SYSNAME NOT NULL
);

-------------------------------------------------------------------------------
-- ADD YOUR TARGET TABLES HERE
-------------------------------------------------------------------------------
INSERT INTO #TargetStatsTables (database_name, schema_name, table_name)
VALUES
   (''YourDB1'', ''dbo'', ''YourTable1''),
   (''YourDB1'', ''dbo'', ''YourTable2''),
   (''YourDB2'', ''sales'', ''Orders'');
-------------------------------------------------------------------------------

IF OBJECT_ID(''tempdb..#StatsRunResults'') IS NOT NULL
   DROP TABLE #StatsRunResults;

CREATE TABLE #StatsRunResults
(
   log_id          INT IDENTITY(1,1) PRIMARY KEY,
   database_name     SYSNAME,
   schema_name       SYSNAME,
   table_name        SYSNAME,
   update_mode       VARCHAR(10),
   command_text      NVARCHAR(MAX),
   start_time        DATETIME2(0),
```

```
    end_time        DATETIME2(0),
    duration_seconds  INT NULL,
    status          VARCHAR(20),
    error_message     NVARCHAR(4000) NULL
);

DECLARE
    @database_name SYSNAME,
    @schema_name   SYSNAME,
    @table_name    SYSNAME,
    @sql           NVARCHAR(MAX);

DECLARE target_cursor CURSOR LOCAL FAST_FORWARD FOR
    SELECT database_name, schema_name, table_name
    FROM #TargetStatsTables
    ORDER BY database_name, schema_name, table_name;

OPEN target_cursor;
FETCH NEXT FROM target_cursor INTO @database_name, @schema_name, @table_name;

WHILE @@FETCH_STATUS = 0
BEGIN
    BEGIN TRY
        SET @sql =
            N''USE '' + QUOTENAME(@database_name) + N'';
              UPDATE STATISTICS ''
            + QUOTENAME(@schema_name) + N''.'' + QUOTENAME(@table_name) + N'' ''
            + @UpdateClause + N'';'';

        DECLARE @start_time DATETIME2(0) = SYSDATETIME();

        EXEC sys.sp_executesql @sql;

        INSERT INTO #StatsRunResults
        (
            database_name, schema_name, table_name, update_mode,
            command_text, start_time, end_time, duration_seconds, status, error_message
        )
        VALUES
        (
            @database_name, @schema_name, @table_name, @StatsMode,
            @sql, @start_time, SYSDATETIME(),
            DATEDIFF(SECOND, @start_time, SYSDATETIME()),
            ''SUCCESS'', NULL
        );
    END TRY
    BEGIN CATCH
        INSERT INTO #StatsRunResults
        (
            database_name, schema_name, table_name, update_mode,
            command_text, start_time, end_time, duration_seconds, status, error_message
        )
        VALUES
        (
            @database_name, @schema_name, @table_name, @StatsMode,
            @sql, SYSDATETIME(), SYSDATETIME(), NULL,
            ''FAILED'', ERROR_MESSAGE()
        );
    END CATCH;

    FETCH NEXT FROM target_cursor INTO @database_name, @schema_name, @table_name;
END;

CLOSE target_cursor;
DEALLOCATE target_cursor;
```

```
SELECT *
FROM #StatsRunResults
ORDER BY database_name, schema_name, table_name;
';
GO

-------------------------------------------------------------------------------
-- Add Schedule
-- Monday-Saturday at 1:30 AM
-------------------------------------------------------------------------------
EXEC msdb.dbo.sp_add_schedule
   @schedule_name = N'Daily Update Statistics - First Pass Schedule',
   @enabled = 1,
   @freq_type = 8,            -- weekly
   @freq_interval = 126,      -- Mon-Sat (2+4+8+16+32+64)
   @freq_recurrence_factor = 1,
   @active_start_date = 20260409,
   @active_end_date = 99991231,
   @active_start_time = 013000,
   @active_end_time = 235959;
GO

-------------------------------------------------------------------------------
-- Attach Schedule
-------------------------------------------------------------------------------
EXEC msdb.dbo.sp_attach_schedule
   @job_name = N'Daily Update Statistics - First Pass',
   @schedule_name = N'Daily Update Statistics - First Pass Schedule';
GO

-------------------------------------------------------------------------------
-- Add job to local server
-------------------------------------------------------------------------------
EXEC msdb.dbo.sp_add_jobserver
   @job_name = N'Daily Update Statistics - First Pass',
   @server_name = N'(LOCAL)';
GO

COMMIT TRANSACTION;
GO
```

b.) Monthly Statistics Update Job Script

```
USE msdb;
GO

SET NOCOUNT ON;
GO

-------------------------------------------------------------------------------
-- Drop existing job if it already exists
-------------------------------------------------------------------------------
IF EXISTS
(
   SELECT 1
   FROM msdb.dbo.sysjobs
   WHERE name = N'Monthly FULLSCAN Update Statistics - First Pass'
)
BEGIN
   EXEC msdb.dbo.sp_delete_job
      @job_name = N'Monthly FULLSCAN Update Statistics - First Pass',
      @delete_unused_schedule = 1;
END
```

```
GO

BEGIN TRANSACTION;
GO

DECLARE @ReturnCode INT = 0;
DECLARE @JobId BINARY(16);
GO

-----------------------------------------------------------------------
-- Create Job
-----------------------------------------------------------------------
EXEC msdb.dbo.sp_add_job
   @job_name = N'Monthly FULLSCAN Update Statistics - First Pass',
   @enabled = 1,
   @notify_level_eventlog = 0,
   @notify_level_email = 0,
   @notify_level_netsend = 0,
   @notify_level_page = 0,
   @delete_level = 0,
   @description = N'Performs first-pass targeted monthly statistics maintenance using table-level UPDATE STATISTICS WITH
FULLSCAN against a curated list of tables. Intended for higher-accuracy refresh on selected objects that justify deeper statistics
collection.',
   @category_name = N'[Uncategorized (Local)]',
   @owner_login_name = SUSER_SNAME(),
   @job_id = @JobId OUTPUT;
GO

-----------------------------------------------------------------------
-- Add Job Step
-----------------------------------------------------------------------
EXEC msdb.dbo.sp_add_jobstep
   @job_name = N'Monthly FULLSCAN Update Statistics - First Pass',
   @step_name = N'Run targeted FULLSCAN stats updates',
   @subsystem = N'TSQL',
   @database_name = N'master',
   @on_success_action = 1,
   @on_fail_action = 2,
   @retry_attempts = 0,
   @retry_interval = 0,
   @command = N'
SET NOCOUNT ON;

DECLARE @StatsMode VARCHAR(10) = ''MONTHLY'';
DECLARE @UpdateClause NVARCHAR(100);

SET @UpdateClause =
   CASE
      WHEN @StatsMode = ''DAILY'' THEN N''WITH SAMPLE 10 PERCENT''
      WHEN @StatsMode = ''MONTHLY'' THEN N''WITH FULLSCAN''
      ELSE NULL
   END;

IF @UpdateClause IS NULL
BEGIN
   RAISERROR(''Invalid @StatsMode. Use DAILY or MONTHLY.'', 16, 1);
   RETURN;
END;

IF OBJECT_ID(''tempdb..#TargetStatsTables'') IS NOT NULL
   DROP TABLE #TargetStatsTables;

CREATE TABLE #TargetStatsTables
(
   database_name SYSNAME NOT NULL,
```

```
    schema_name   SYSNAME NOT NULL,
    table_name    SYSNAME NOT NULL
);

-------------------------------------------------------------------------------
-- ADD YOUR TARGET TABLES HERE
-------------------------------------------------------------------------------
INSERT INTO #TargetStatsTables (database_name, schema_name, table_name)
VALUES
    (''YourDB1'', ''dbo'', ''YourTable1''),
    (''YourDB1'', ''dbo'', ''YourTable2''),
    (''YourDB2'', ''sales'', ''Orders'');
-------------------------------------------------------------------------------

IF OBJECT_ID(''tempdb..#StatsRunResults'') IS NOT NULL
    DROP TABLE #StatsRunResults;

CREATE TABLE #StatsRunResults
(
    log_id          INT IDENTITY(1,1) PRIMARY KEY,
    database_name     SYSNAME,
    schema_name       SYSNAME,
    table_name        SYSNAME,
    update_mode       VARCHAR(10),
    command_text      NVARCHAR(MAX),
    start_time        DATETIME2(0),
    end_time          DATETIME2(0),
    duration_seconds  INT NULL,
    status            VARCHAR(20),
    error_message     NVARCHAR(4000) NULL
);

DECLARE
    @database_name SYSNAME,
    @schema_name   SYSNAME,
    @table_name    SYSNAME,
    @sql           NVARCHAR(MAX);

DECLARE target_cursor CURSOR LOCAL FAST_FORWARD FOR
    SELECT database_name, schema_name, table_name
    FROM #TargetStatsTables
    ORDER BY database_name, schema_name, table_name;

OPEN target_cursor;
FETCH NEXT FROM target_cursor INTO @database_name, @schema_name, @table_name;

WHILE @@FETCH_STATUS = 0
BEGIN
    BEGIN TRY
        SET @sql =
            N''USE '' + QUOTENAME(@database_name) + N'';
              UPDATE STATISTICS ''
            + QUOTENAME(@schema_name) + N''.'' + QUOTENAME(@table_name) + N'' ''
            + @UpdateClause + N'';'';

        DECLARE @start_time DATETIME2(0) = SYSDATETIME();

        EXEC sys.sp_executesql @sql;

        INSERT INTO #StatsRunResults
        (
            database_name, schema_name, table_name, update_mode,
            command_text, start_time, end_time, duration_seconds, status, error_message
        )
        VALUES
```

```
      (
        @database_name, @schema_name, @table_name, @StatsMode,
        @sql, @start_time, SYSDATETIME(),
        DATEDIFF(SECOND, @start_time, SYSDATETIME()),
        ''SUCCESS'', NULL
      );
    END TRY
    BEGIN CATCH
      INSERT INTO #StatsRunResults
      (
        database_name, schema_name, table_name, update_mode,
        command_text, start_time, end_time, duration_seconds, status, error_message
      )
      VALUES
      (
        @database_name, @schema_name, @table_name, @StatsMode,
        @sql, SYSDATETIME(), SYSDATETIME(), NULL,
        ''FAILED'', ERROR_MESSAGE()
      );
    END CATCH;

    FETCH NEXT FROM target_cursor INTO @database_name, @schema_name, @table_name;
END;

CLOSE target_cursor;
DEALLOCATE target_cursor;

SELECT *
FROM #StatsRunResults
ORDER BY database_name, schema_name, table_name;
';
GO

-------------------------------------------------------------------------------
-- Add Schedule
-- Last Sunday of the month at 1:00 AM
-------------------------------------------------------------------------------
EXEC msdb.dbo.sp_add_schedule
    @schedule_name = N'Monthly FULLSCAN Update Statistics - First Pass Schedule',
    @enabled = 1,
    @freq_type = 32,              -- monthly relative
    @freq_interval = 1,           -- Sunday
    @freq_relative_interval = 16,    -- Last
    @freq_recurrence_factor = 1,
    @active_start_date = 20260409,
    @active_end_date = 99991231,
    @active_start_time = 010000,
    @active_end_time = 235959;
GO

-------------------------------------------------------------------------------
-- Attach Schedule
-------------------------------------------------------------------------------
EXEC msdb.dbo.sp_attach_schedule
    @job_name = N'Monthly FULLSCAN Update Statistics - First Pass',
    @schedule_name = N'Monthly FULLSCAN Update Statistics - First Pass Schedule';
GO

-------------------------------------------------------------------------------
-- Add job to local server
-------------------------------------------------------------------------------
EXEC msdb.dbo.sp_add_jobserver
    @job_name = N'Monthly FULLSCAN Update Statistics - First Pass',
    @server_name = N'(LOCAL)';
GO
```

```
COMMIT TRANSACTION;
GO
```

After these are setup you'll want to keep an eye on TempDB usage and for any job failures for any issues. If TempDB pressure continues, research your lists to make sure the passes aren't too large, if so you may need to shorten them or possibly break them up into a separate pass job (or add them to an existing pass job). Also evaluate general SQL server usage around the time of the Maintenance Job runs, as there could be other TempDB or other activities that could play a factor in performance and you may need to adjust your job schedule times to compensate.

This approach provides a balanced statistics maintenance strategy that prioritizes high-impact tables, reduces unnecessary system load, and improves overall maintenance efficiency in environments with large or heavily utilized databases.